SQL - 10 queries

1. DEPARTMENT SALARY SUMMARY

```
SELECT
   d.dept_name,
   COUNT(e.emp_id) AS employee_count,
   ROUND(AVG(e.salary), 2) AS avg_salary,
   MIN(e.salary) AS min_salary,
   MAX(e.salary) AS max_salary
FROM Departments d
LEFT JOIN Employees e
   ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
```

2. SECOND HIGHEST SALARY PER DEPARTMENT

```
SELECT emp_id, emp_name, dept_id, salary
FROM (
   SELECT emp_id, emp_name, dept_id, salary,
        DENSE_RANK() OVER (
           PARTITION BY dept_id
           ORDER BY salary DESC
        ) AS salary_rank
   FROM Employees
) AS ranked
WHERE salary_rank = 2;
```

3. EMPLOYEES ABOVE DEPARTMENT AVERAGE

```
SELECT e.emp_id, e.emp_name, e.dept_id, e.salary
FROM Employees e
WHERE e.salary > (
   SELECT AVG(e2.salary)
   FROM Employees e2
   WHERE e2.dept_id = e.dept_id
);
```

4. MANAGER VS TEAM SALARY

```
SELECT
   m.emp_id AS manager_id,
   m.emp_name AS manager_name,
   m.salary AS manager_salary,
   COUNT(e.emp_id) AS direct_report_count,
   SUM(e.salary) AS team_salary
FROM Employees m
```

```
JOIN Employees e
  ON e.manager_id = m.emp_id
GROUP BY m.emp_id, m.emp_name, m.salary
HAVING COUNT(e.emp_id) >= 2;
```

5. TOP PROJECT CONTRIBUTORS

```
SELECT project_id, emp_id, emp_name, total_hours
FROM (
  SELECT *,
      DENSE_RANK() OVER (
        PARTITION BY project_id
        ORDER BY total_hours DESC
      ) AS contributor_rank
  FROM (
    SELECT
      ep.project_id,
      ep.emp_id,
      e.emp_name,
      SUM(ep.hours_worked) AS total_hours
    FROM EmployeeProjects ep
    JOIN Employees e
      ON ep.emp_id = e.emp_id
    GROUP BY ep.project_id, ep.emp_id, e.emp_name
  ) AS project_hours
) AS ranked
WHERE contributor_rank <= 2;
```

6. DEPARTMENTS WITHOUT PROJECTS

```
SELECT d.dept_id, d.dept_name
FROM Departments d
WHERE EXISTS (
  SELECT 1
  FROM Employees e
  WHERE e.dept_id = d.dept_id
)
AND NOT EXISTS (
  SELECT 1
  FROM Projects p
  WHERE p.dept_id = d.dept_id
);
```

7. RUNNING ORDER REVENUE

```
SELECT
  order_day,
  daily_revenue,
  SUM(daily_revenue) OVER (
    ORDER BY order_day
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS cumulative_revenue
FROM (
  SELECT
    order_date::date AS order_day,
    SUM(amount) AS daily_revenue
  FROM Orders
  WHERE status = 'completed'
  GROUP BY order_date::date
) AS daily_orders
ORDER BY order_day;
```

8. CUSTOMER ORDER CLASSIFICATION

```
SELECT
  customer_id,
  total_amount,
  CASE
    WHEN total_amount >= 100000 THEN 'PLATINUM'
    WHEN total_amount >= 50000 THEN 'GOLD'
    WHEN total_amount >= 20000 THEN 'SILVER'
    ELSE 'BRONZE'
  END AS category
FROM (
  SELECT
    customer_id,
    SUM(amount) AS total_amount
  FROM Orders
  WHERE status = 'completed'
  GROUP BY customer_id
) AS CustomerRevenue;
```

DSA CODING QUESTIONS

1. LONGEST SUBARRAY WITH SUM K

```
static int longestSubarray(int[] arr, int k) {
    Map<Long, Integer> map = new HashMap<>();
    long sum = 0;
```

```
    int maxLen = 0;
    map.put(0L, -1);

    for (int i = 0; i < arr.length; i++) {
        sum += arr[i];

        if (map.containsKey(sum - k)) {
            maxLen = Math.max(maxLen,
                i - map.get(sum - k));
        }

        map.putIfAbsent(sum, i);
    }

    return maxLen;
}
```

2. MINIMUM WINDOW CONTAINING ALL REQUIRED SKILLS

```
static String minWindow(String s, String t) {
    if (t.isEmpty() || s.length() < t.length())
        return "";

    Map<Character, Integer> need = new HashMap<>();
    Map<Character, Integer> window = new HashMap<>();

    for (char c : t.toCharArray())
        need.put(c, need.getOrDefault(c, 0) + 1);

    int left = 0, formed = 0, required = need.size();
    int start = 0, minLen = Integer.MAX_VALUE;
```

```
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        window.put(c, window.getOrDefault(c, 0) + 1);

        if (need.containsKey(c)
            && window.get(c).intValue() == need.get(c).intValue())
            formed++;

        while (formed == required) {
            if (right - left + 1 < minLen) {
                minLen = right - left + 1;
                start = left;
            }

            char removed = s.charAt(left++);
            window.put(removed, window.get(removed) - 1);

            if (need.containsKey(removed)
                && window.get(removed) < need.get(removed))
                formed--;
        }
    }

    return minLen == Integer.MAX_VALUE
        ? "" : s.substring(start, start + minLen);
}
```

3. MERGE OVERLAPPING INTERVALS

```
static int[][] merge(int[][] intervals) {
    if (intervals.length == 0)
```

```
        return new int[0][2];

    Arrays.sort(intervals,
        (a, b) -> Integer.compare(a[0], b[0]));

    List<int[]> result = new ArrayList<>();
    int start = intervals[0][0];
    int end = intervals[0][1];

    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] <= end) {
            end = Math.max(end, intervals[i][1]);
        } else {
            result.add(new int[]{start, end});
            start = intervals[i][0];
            end = intervals[i][1];
        }
    }

    result.add(new int[]{start, end});
    return result.toArray(new int[result.size()][]);
}
```

5. DETECT AND REMOVE CYCLE IN LINKED LIST

```
static boolean removeCycle(Node head) {
    if (head == null || head.next == null)
        return false;

    Node slow = head, fast = head;
```

```
    do {
        if (fast == null || fast.next == null)
            return false;

        slow = slow.next;
        fast = fast.next.next;
    } while (slow != fast);

    slow = head;

    if (slow == fast) {
        while (fast.next != slow)
            fast = fast.next;
    } else {
        while (slow.next != fast.next) {
            slow = slow.next;
            fast = fast.next;
        }
    }

    fast.next = null;
    return true;
}
```

6. LOWEST COMMON ANCESTOR IN BINARY TREE

```
static Node lca(Node root, Node p, Node q) {
    if (root == null || root == p || root == q)
        return root;

    Node left = lca(root.left, p, q);
```

```
    Node right = lca(root.right, p, q);

    if (left != null && right != null)
        return root;

    return left != null ? left : right;
}
```

7. SHORTEST PATH IN A BINARY MATRIX

```
static int shortestPath(int[][] grid) {
    int n = grid.length;

    if (n == 0 || grid[0].length != n)
        return -1;

    if (grid[0][0] != 0 || grid[n - 1][n - 1] != 0)
        return -1;

    int[][] dirs = {
        {-1,-1}, {-1,0}, {-1,1},
        {0,-1},          {0,1},
        {1,-1},  {1,0},   {1,1}
    };

    Queue<int[]> queue = new LinkedList<>();
    boolean[][] visited = new boolean[n][n];

    queue.offer(new int[]{0, 0, 1});
    visited[0][0] = true;
```

```
    while (!queue.isEmpty()) {
        int[] cur = queue.poll();
        int r = cur[0], c = cur[1], dist = cur[2];

        if (r == n - 1 && c == n - 1)
            return dist;

        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];

            if (nr >= 0 && nr < n && nc >= 0 && nc < n
                && grid[nr][nc] == 0 && !visited[nr][nc]) {
                visited[nr][nc] = true;
                queue.offer(new int[]{nr, nc, dist + 1});
            }
        }
    }

    return -1;
}
```

9. MINIMUM COINS FOR AMOUNT

```
static int coinChange(int[] coins, int amount) {
    if (amount < 0)
        return -1;

    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;
```

```
    for (int a = 1; a <= amount; a++) {
        for (int coin : coins) {
            if (coin > 0 && coin <= a)
                dp[a] = Math.min(dp[a], dp[a - coin] + 1);
        }
    }

    return dp[amount] > amount ? -1 : dp[amount];
}
```